

DiM-WAM: World-Action Modeling with Diverse Historical Event Memory

Kai Wang^{1,2}, Zhaopeng Gu^{1,2}, Yixiang Chen¹, Yuan Xu¹, Qisen Ma¹,
Zhaowen Li^{2,*}, Yan Huang^{1,3,*}, Liang Wang¹, Peng Su²,

Abstract—World-action models have shown promising robot-manipulation performance by jointly predicting future visual states and actions. However, existing methods mainly rely on short-term history and short-horizon future prediction, which is insufficient for long-horizon tasks whose correct execution depends on earlier observations and task progress. To address long-term forgetting and poor awareness of the global task state, we introduce DiM-WAM, a memory-augmented world-action model that integrates multi-scale historical context, local future dynamics, and global task progress. Learned bank-specific queries combine with bank identity embeddings to extract complementary observation summaries from each real observation. Each summary is then compressed within its corresponding memory bank by a shared similarity-based update rule, and the retained bank tokens jointly condition video and action denoising through slot-relative temporal positions. A progress-supervision objective further encourages memory tokens to encode not only completed historical events but also the current task stage and its implications for the remaining task. In our controlled WAM comparison, which fixes the LingBot-VA backbone, local inputs, demonstrations, audited camera/window configuration, and common training configuration apart from the proposed memory/progress components, DiM-WAM improves RMBench average success from 34.8% to 69.8%. On four real-world Franka tasks under the same few-shot setup, it improves average stage success from 70.7% to 91.5% and full-task success from 52.5% to 80.0%. Project page: <https://wangkai-casia.github.io/dim-wam/>.

I. INTRODUCTION

Vision-language-action (VLA) models learn semantic action mappings from observations and instructions [1]–[5]. Because action labels describe contact and scene evolution only indirectly, world-action models (WAMs) additionally predict future visual states and use visual change as dense supervision for action-conditioned dynamics [6]–[8]. Existing WAMs are often instantiated with short local contexts and short prediction horizons, a design that becomes limiting when the current observation does not uniquely determine the correct action.

Consider filling a cup at a water dispenser. When the robot reaches for the same switch, observations before turning the water on and before turning it off can be nearly identical. Without remembering whether it has already activated the dispenser, a local policy may repeatedly toggle the switch or fail to press it when required. This failure raises two questions: how should a WAM retain factual evidence beyond its local

context, and how should it organize that evidence under fixed capacity?

Neither a longer prediction horizon nor a larger input window fully resolves the problem. Predicted futures cannot recover an earlier observation and should not be written into persistent memory, where generation errors could propagate across later decisions. A larger window increases computation and forces sparse early evidence to compete with many locally redundant frames. Long-horizon WAMs therefore need bounded memory that persists beyond the local window and is corrected by real observations during closed-loop execution.

A single bounded bank introduces a second failure mode. Initial configurations, completed subtask outcomes, and recent changes have different retention horizons and update frequencies, yet compete for the same slots. Frequent local changes may overwrite sparse early evidence, while preserving too much early evidence can reduce sensitivity to new outcomes. This engineering tension motivates exploring parallel banks. The multiple-memory-subsystems framework provides an analogy in which an experience can support complementary representations across interacting subsystems [9]; we use this analogy only as design inspiration, not as a biological model or evidence of architectural optimality.

Recent memory-augmented VLA methods use explicit banks, perceptual-cognitive memory, multiscale memory, or recurrent queries [10]–[13]. These methods mainly condition action prediction. Memory in a WAM must additionally condition future visual generation while remaining grounded in real environmental feedback.

Accordingly, we propose DiM-WAM, a memory-augmented WAM with Diverse Historical Event Memory (DHEM). Multiple fixed-capacity banks use bank-specific queries and identity embeddings to extract complementary summaries from each real observation, then compress them independently. Valid tokens are flattened with slot-relative temporal positions and jointly condition video and action denoising. Persistent memory is updated only after real feedback, separating observed history from generated hypotheses, while an auxiliary progress objective encourages retained events to encode a coarse task-progress proxy. Together, the design covers local continuity, immediate future dynamics, cross-stage factual evidence, and coarse task-stage information.

Experiments on RMBench [14] and real Franka Panda tasks support this design. Under the RMBench task environments and success metrics, DiM-WAM reaches 69.8% average suc-

¹CASIA. ²Yinwang Intelligent Technology. ³FiveAges.

*Corresponding authors: Zhaowen Li lizhaowen@yinwang.com; Yan Huang yhuang@nlpr.ia.ac.cn.

cess, the highest among the listed methods; the benchmark-reported Mem-0 baseline reaches 42.0% under a possibly different training recipe. Because baseline families may use different training recipes and model interfaces, we reserve causal attribution for the controlled WAM comparison with LingBot-VA [7], which fixes the backbone, inputs, demonstrations, audited camera/window setting, and common training configuration.

Under this controlled setup, DiM-WAM improves RM-Bench success from 34.8% to 69.8%. On real tasks, stage success rises from 70.7% to 91.5%, and full-task success from 52.5% to 80.0%. Ablations indicate the roles of multi-bank memory and progress supervision, while cross-bank analysis qualitatively suggests complementary event selection and representation subspaces.

II. RELATED WORK

A. Vision-Language-Action Models and Temporal Context

Vision-language-action (VLA) models have advanced robot policy learning through large-scale semantic transfer, continuous action generation, and spatial representation learning [1]–[5], [15], [16]. However, partially observable and non-Markovian manipulation requires policies to use temporal context beyond the current observation.

Recent memory-augmented VLA methods address this issue by introducing explicit memory banks, perceptual-cognitive memories, multiscale embodied memories, recurrent queries, sparse visual evidence, or joint history-future context tokens [10]–[13], [17], [18]. These designs show that multiscale decomposition, selective writing, and long-term compression can improve history utilization.

Nevertheless, these memory modules mainly expose action prediction as the final interface. They rarely study how memory should participate in autoregressive visual-state updates or how closed-loop real observations can correct generation-induced state bias. This distinction is central to memory-augmented long-horizon WAMs.

B. World Action Models

Early video-based policy learning formulates decision making as conditional future-video generation and then recovers actions from generated visual trajectories [19]–[21]. Recent world-action models (WAMs) move toward joint modeling of visual dynamics and actions, using future prediction as dense dynamic supervision for policy learning [6]–[8], [22], [23].

Several WAMs further unify action, future visual/proprioceptive states, and value prediction under diffusion or cascaded video-action modeling objectives [24], [25]. Others improve efficiency by compressing future observations, decoupling joint video training from test-time generation, or decoding only actions at inference [26]–[28].

Despite these advances, existing WAMs mainly address local future modeling, training efficiency, or short-horizon closed-loop control. Cross-stage historical-state management remains underexplored. Our work studies a complementary problem: how to build a compressible, retrievable, and

observation-corrected long-term memory for WAMs so that future prediction and action decisions remain consistent during long-horizon execution.

C. Memory-Augmented Long Video Generation

Long-video generation extends fixed-length video models autoregressively, typically with local temporal windows and additional mechanisms for long-range consistency [29]. To reduce long-context cost, recent methods replace global attention with state-space models, block scans, or linear attention, enabling longer generation with near-linear or constant memory growth [30]–[32].

Another line compresses history through recurrent memory vectors, KV-compression, hybrid local-global caches, or training objectives designed to reduce forgetting and drift [33]–[36]. More structured approaches partition history by function, dynamically retrieve relevant clips, or maintain entity-centric memories for characters, objects, backgrounds, and temporal slots [37]–[40]. These results suggest that memory organization, rather than capacity alone, is crucial for long-range generation.

However, long-video generation mainly targets appearance, identity, and narrative consistency. Long-horizon robot control additionally requires memory to represent task progress, object states, and action consequences, while being corrected by real observations at each step. We therefore introduce structured memory into WAMs, using multiple memory banks to carry information with different temporal scales and functional roles while jointly supporting future prediction and action decisions.

III. METHOD

A. Overview

Building on the Introduction, we posit two hypotheses: long-horizon decisions benefit from complementary temporal signals, and heterogeneous history is better represented by parallel banks than by a monolithic memory. For decision segment i , we represent these complementary sources as

$$\mathcal{I}_i = \left(\mathcal{H}_i^{\text{long}}, \mathcal{H}_i^{\text{short}}, \mathcal{F}_i^{\text{short}}, \mathcal{G}_i^{\text{prog}} \right). \quad (1)$$

Here, $\mathcal{H}_i^{\text{short}}$ and $\mathcal{H}_i^{\text{long}}$ denote recent and cross-stage history, respectively. $\mathcal{F}_i^{\text{short}}$ is the predicted local video-action evolution, whereas $\mathcal{G}_i^{\text{prog}}$ is a coarse trajectory-progress proxy rather than an explicit future rollout.

Accordingly, multi-scale world-action modeling combines local history and future prediction with cross-stage memory and progress supervision. Following the MMSS-inspired organization principle, DHEM uses parallel banks that compress independently and read cooperatively; this is architectural inspiration rather than a biological model.

As shown in Fig. 1, the model reads memory and local context, predicts short-term video and actions, executes the latter, and updates memory only from the resulting real observation. This closed loop prevents generated errors from entering persistent history.

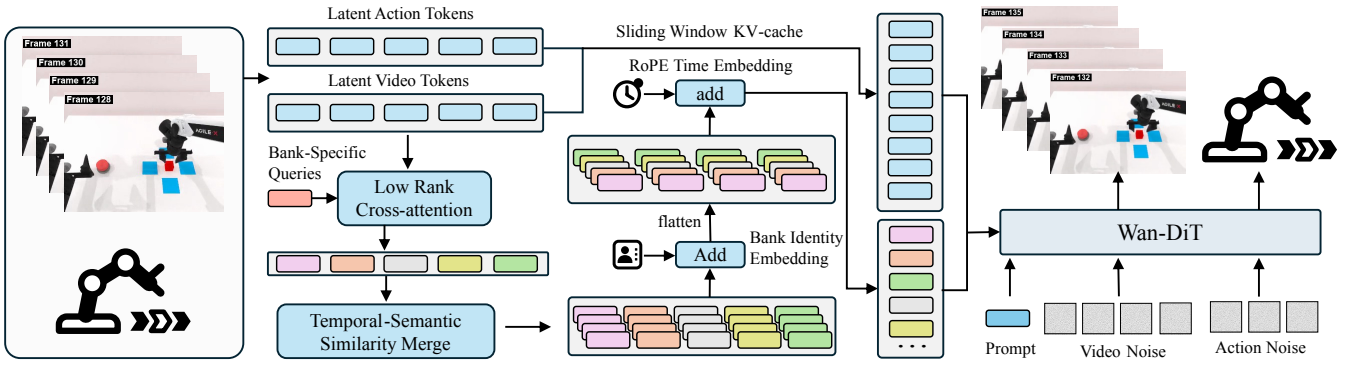


Fig. 1. Overview of DiM-WAM. Bank-specific queries combine with bank identity embeddings to extract complementary summaries from each real observation, and each bank compresses its summary against its own retained state. Valid tokens are flattened in slot-major order, assigned slot-relative RoPE positions, and jointly read to predict short-term video and actions.

B. Multi-Scale World-Action Modeling

To address information coverage, let c be the instruction, $\mathcal{C}_i$ the local WAM context, and $\mathcal{M}_i$ the long-term memory. For horizon H , the model predicts visual latents and actions as

$$(\hat{\mathbf{z}}_{i:i+H-1}, \hat{\mathbf{a}}_{i:i+H-1}) = f_{\theta}(\mathcal{C}_i, c; \mathcal{M}_i). \quad (2)$$

$\mathcal{C}_i$ combines a sliding window and key-value (KV) cache for recent motion continuity, but cannot reliably distinguish visually similar states at different task stages. $\mathcal{M}_i$ supplies cross-stage evidence, while progress supervision encourages coarse task-stage information. In the notation of Eq. (1), $\mathcal{H}_i^{\text{short}}$ and $\mathcal{F}_i^{\text{short}}$ are jointly realized through the sliding-window context $\mathcal{C}_i$ in Eq. (2), $\mathcal{H}_i^{\text{long}}$ corresponds to the persistent memory $\mathcal{M}_i$, and $\mathcal{G}_i^{\text{prog}}$ is instantiated indirectly by the progress-supervision objective in Sec. III-D.

C. DHEM Design

a) *Memory structure.*: Following the subsystem-inspired principle, we treat initial states, completed subtask outcomes, and recent changes as events with different semantics, retention horizons, and update frequencies. Because one bounded sequence forces them to compete, we use K parallel banks:

$$\begin{aligned} \mathcal{M}_i &= \{\mathcal{M}_i^k\}_{k=1}^K, \\ \mathcal{M}_i^k &= \{e_{i,k,n}\}_{n=1}^N, \\ e_{i,k,n} &= (\mathbf{m}_{i,k,n}, t_{i,k,n}, \alpha_{i,k,n}, v_{i,k,n}). \end{aligned} \quad (3)$$

Each slot stores a token $\mathbf{m}$, time t , accumulated mass α , and validity flag v . Thus, memory has a fixed capacity of KN slots. Bank roles are not predefined. Learned bank-specific extraction queries and bank identity embeddings jointly break permutation symmetry at writing time, while independent bank states and the training objectives promote complementary representations.

b) *Visual event extraction.*: After executing segment i , the real observation is encoded into visual features $\mathbf{X}_i = [\mathbf{x}_{i,j}]_{j=1}^{L_i}$. Each bank k has a learned summary query $\mathbf{q}_k$

and bank embedding $\mathbf{b}_k$. The bank-conditioned query and normalized observation features are

$$\mathbf{h}_{i,j} = \text{LN}_x(\mathbf{x}_{i,j}), \quad \mathbf{r}_k = \text{LN}_q(\mathbf{q}_k + \mathbf{b}_k). \quad (4)$$

Using shared query and key projections of rank d_r , bank k extracts

$$\begin{aligned} a_{i,k,j} &= \text{Softmax}_j \left(\frac{(W_q \mathbf{r}_k)^\top (W_h \mathbf{h}_{i,j})}{\sqrt{d_r}} \right), \\ \mathbf{s}_{i,k} &= \sum_{j=1}^{L_i} a_{i,k,j} \mathbf{h}_{i,j}, \\ \tilde{\mathbf{u}}_{i,k} &= \mathbf{s}_{i,k} + \text{FFN}(\text{LN}_o(\mathbf{s}_{i,k})). \end{aligned} \quad (5)$$

Thus, all banks observe the same real feature sequence but receive different learned summaries $\tilde{\mathbf{u}}_{i,k}$. The identity embedding $\mathbf{b}_k$ explicitly conditions what bank k extracts from the observation and is therefore part of the persistent write path. The persistent state stores bank-conditioned observation summaries, not generated future predictions.

c) *Bounded event compression.*: In each bank, slot 1 is a fixed initial-state anchor, slot N is the latest event, and slots $2, \dots, N-1$ contain compressed history. For clarity, we omit segment and bank indices below.

The priority for merging two valid events p and q is

$$\rho_k(p, q) = \frac{1 + \cos(\mathbf{m}_p, \mathbf{m}_q)}{2} \exp\left(-\frac{|t_p - t_q|}{\tau}\right), \quad (6)$$

where τ controls temporal decay. High priority therefore indicates semantic redundancy at nearby times.

When a bank is full, only adjacent middle-history pairs are eligible:

$$\mathcal{A}_{i,k} = \{(s, s+1) \mid 2 \leq s \leq N-2, v_{k,s} v_{k,s+1} = 1\}. \quad (7)$$

Let

$$\begin{aligned} (p^*, q^*) &= \arg \max_{(p,q) \in \mathcal{A}_{i,k}} \rho_k(p, q), \\ \rho_{i,k}^{\text{new}} &= \frac{1 + \cos(\mathbf{m}_{k,N}, \tilde{\mathbf{u}}_{i,k})}{2} \exp\left(-\frac{|t_{k,N} - i|}{\tau}\right). \end{aligned} \quad (8)$$

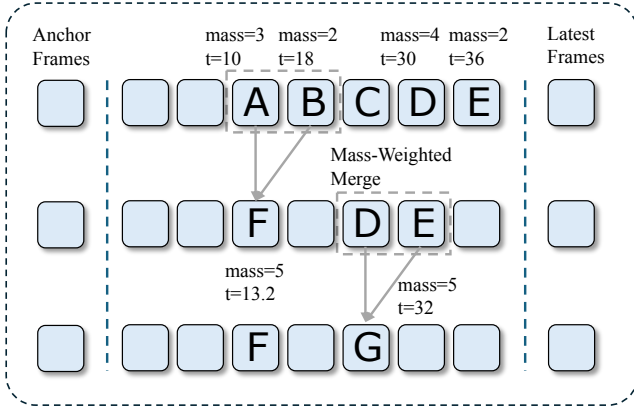


Fig. 2. Within-bank similarity-based merging under fixed capacity. At each update, the highest-priority adjacent pair in the middle history is replaced by a mass-weighted representative whose accumulated mass and timestamp are updated by Eq. (10). The initial anchor and latest event are excluded from merging.

The candidate is discarded when

$$\rho_{i,k}^{\text{new}} \geq \rho_k(p^*, q^*), \quad (9)$$

because it is at least as redundant with the latest event as the most mergeable historical pair. Otherwise, (p^*, q^*) is merged to make room for the candidate.

The merged event preserves accumulated evidence through

$$\mathbf{m}_{pq} = \frac{\alpha_p \mathbf{m}_p + \alpha_q \mathbf{m}_q}{\alpha_p + \alpha_q}, \quad t_{pq} = \frac{\alpha_p t_p + \alpha_q t_q}{\alpha_p + \alpha_q}, \quad (10)$$

$$\alpha_{pq} = \alpha_p + \alpha_q.$$

Fig. 2 illustrates successive within-bank updates: each selected adjacent pair is replaced by a mass-weighted representative, while the initial anchor and latest-event slots remain protected.

Algorithm 1 summarizes one bank update. We use the same similarity function and $\tau = N - 1$ in every bank. The banks nevertheless follow different trajectories because Eq. (5) supplies bank-specific candidates and each update depends on the bank’s own retained state. For tensor initialization, all slots are filled with the first-frame anchor; these duplicate middle anchors are explicitly evicted before the normal similarity-based rule is applied.

Each bank applies this rule to its own retained history and bank-conditioned candidate. Memory cost remains $O(KN)$, independent of trajectory length.

d) Cooperative reading.: Before segment i , valid memory tokens are flattened in slot-major, bank-minor order. Bank identity is already encoded into persistent tokens by the write-time embedding $\mathbf{b}_k$ in Eq. (5), so no additional bank embedding is added during reading. For zero-indexed slot $n \in \{0, \dots, N - 1\}$, the implementation assigns the slot-relative temporal coordinate

$$\pi_{i,n} = \pi_i^{\text{cur}} - 2(N - 1 - n), \quad (11)$$

Algorithm 1 Update of bank k at segment i

Require: Bank $\mathcal{M}^k$, features $\mathbf{X}_i$, query $\mathbf{q}_k$, identity embedding $\mathbf{b}_k$, time i

Ensure: Updated bank $\mathcal{M}^k$

- 1: $\mathbf{r}_k \leftarrow \text{LN}_q(\mathbf{q}_k + \mathbf{b}_k)$
- 2: $\tilde{\mathbf{u}}_{i,k} \leftarrow \text{BankSummarize}(\mathbf{X}_i; \mathbf{r}_k)$ using Eq. (5)
- 3: $e_{\text{new}} \leftarrow (\tilde{\mathbf{u}}_{i,k}, i, 1, 1)$
- 4: **if** $\mathcal{M}^k$ is uninitialized **then**
- 5: Fill all slots with e_{new} ; fix e_1 as the anchor
- 6: **return** $\mathcal{M}^k$
- 7: **end if**
- 8: **if** e_N duplicates the initial anchor **then**
- 9: $e_N \leftarrow e_{\text{new}}$
- 10: **return** $\mathcal{M}^k$
- 11: **end if**
- 12: **if** a middle slot e_r duplicates the initial anchor **then**
- 13: Remove e_r , compact the middle slots, and move e_N to slot $N - 1$
- 14: $e_N \leftarrow e_{\text{new}}$
- 15: **return** $\mathcal{M}^k$
- 16: **end if**
- 17: **if** an invalid middle slot exists **then**
- 18: Move e_N to the first invalid middle slot
- 19: $e_N \leftarrow e_{\text{new}}$
- 20: **return** $\mathcal{M}^k$
- 21: **end if**
- 22: $(p^*, q^*) \leftarrow \arg \max_{(p,q) \in \mathcal{A}_{i,k}} \rho_k(p, q)$
- 23: **if** $\rho_{i,k}^{\text{new}} \geq \rho_k(p^*, q^*)$ **then**
- 24: **return** $\mathcal{M}^k$
- 25: **end if**
- 26: Save e_N and merge e_{p^*}, e_{q^*} using Eq. (10)
- 27: Compact the middle slots in chronological order
- 28: Move the saved latest event to slot $N - 1$
- 29: $e_N \leftarrow e_{\text{new}}$
- 30: **return** $\mathcal{M}^k$

where π_i^{cur} is the temporal coordinate of the current segment. The readout is

$$\mathbf{R}_i = \text{RoPE}_\pi \left(\text{Flat}_{n,k} [(\mathbf{m}_{i,k,n}, \pi_{i,n})]_{v_{i,k,n}=1} \right). \quad (12)$$

This uses slot recency rather than sorting by the mass-weighted timestamps stored for compression. Writing and compression remain bank-local, whereas Transformer attention jointly reads $\mathbf{R}_i$. The same memory sequence conditions video and action denoising, grounding both branches in a common task history.

As illustrated in Fig. 3, memory tokens are temporary attention conditions for the current segment and do not extend the persistent KV cache. They are cleared after prediction; persistent memory is updated only from the subsequent real observation. This separates predictive hypotheses from factual history.

D. Task-Progress-Aware Training

We learn a coarse progress proxy from the memory readout:

$$\mathbf{p}_i = \text{Softmax}(h_{\text{prog}}(\text{Pool}(\mathbf{R}_i))), \quad \mathbf{p}_i \in \Delta^{B-1}. \quad (13)$$

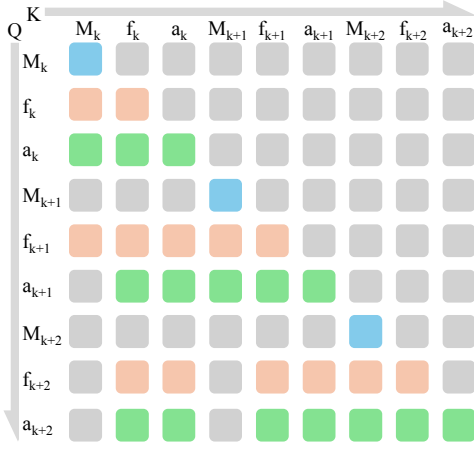


Fig. 3. Visibility among memory, video-latent, and action tokens during denoising. Memory tokens are maintained independently within their respective banks and jointly condition both denoising branches while preserving causal video-action prediction.

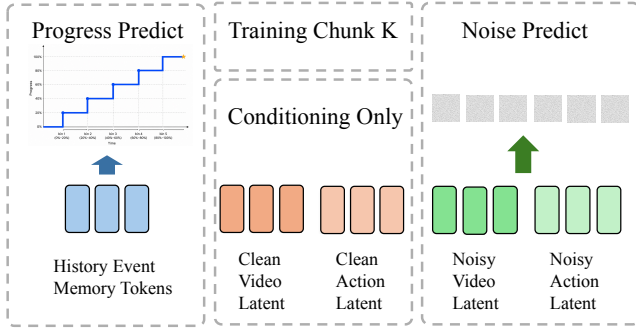


Fig. 4. Progress-proxy supervision encourages memory to encode completed events and coarse task-stage information.

Here, h_{prog} is a prediction head, B is the number of progress bins, and Δ^{B-1} is the probability simplex. We set $B = 10$ in all experiments. For a demonstration with T decision segments indexed by $i \in \{0, \dots, T-1\}$, the normalized completion ratio is

$$r_i = \frac{i}{T-1}. \quad (14)$$

The scalar $r_i \in [0, 1]$ serves only as a coarse progress proxy. The target and loss are

$$y_i = \min(\lfloor Br_i \rfloor, B-1), \quad \mathcal{L}_{\text{prog}} = -\log p_{i,y_i}. \quad (15)$$

The progress head is an auxiliary training objective, not a planner or long-horizon rollout. It encourages retained events to be predictive of coarse trajectory progress, complementing the WAM’s explicit short-horizon video-action prediction.

a) Joint objective.: To discourage bank collapse, let $\mathcal{V}_{i,k} = \{n \mid v_{i,k,n} = 1\}$ and define the normalized bank mean

$$\mu_{i,k} = \frac{1}{|\mathcal{V}_{i,k}|} \sum_{n \in \mathcal{V}_{i,k}} \mathbf{m}_{i,k,n}, \quad \bar{\mathbf{m}}_{i,k} = \frac{\mu_{i,k}}{\|\mu_{i,k}\|_2}. \quad (16)$$

For $\mathcal{P} = \{(k, l) \mid 1 \leq k < l \leq K\}$, the diversity loss is

$$\mathcal{L}_{\text{div}} = \frac{1}{|\mathcal{P}|} \sum_{(k,l) \in \mathcal{P}} (\bar{\mathbf{m}}_{i,k}^\top \bar{\mathbf{m}}_{i,l})^2. \quad (17)$$

Minimizing squared cosine similarity promotes complementary bank representations without assigning semantic roles in advance. It regularizes the distinct views already induced by the learned bank-specific queries and bank identity embeddings; it is not the sole mechanism that breaks bank symmetry.

The complete objective is

$$\mathcal{L} = \mathcal{L}_{\text{video}} + \mathcal{L}_{\text{action}} + \lambda_{\text{div}} \mathcal{L}_{\text{div}} + \lambda_{\text{prog}} \mathcal{L}_{\text{prog}}, \quad (18)$$

The video and action losses have unit weights, while λ_{div} and λ_{prog} balance the two auxiliary objectives. We use $\lambda_{\text{div}} = 10^{-3}$ and $\lambda_{\text{prog}} = 10^{-2}$.

IV. EXPERIMENTS

We evaluate four aspects of DiM-WAM: long-horizon success in simulation and on real robots, the contributions of multi-bank memory and progress supervision, and whether different banks learn complementary behavior.

A. Experimental Setup

a) Base model and benchmark.: We use LingBot-VA [7] as the base WAM. It jointly predicts short-term visual latents and actions from a sliding-window KV cache, but lacks explicit memory once an observation leaves the window. DiM-WAM preserves this interface and adds fixed-capacity long-term memory; both models use identical local inputs.

RMBench [14], built on RoboTwin 2.0, contains five $M(1)$ tasks requiring a few key observations and four $M(n)$ tasks requiring accumulated interaction outcomes. In `put_back_block`, the robot moves a block to the center, presses a button, and returns it to one of four possible initial locations.

As Fig. 5 shows, different initial states converge to similar intermediate observations, while the final target still depends on the initial location. With balanced locations and no initial observation in context, the key target-choice chance is 25%; this is not a full-task upper bound.

b) Implementation details.: RMBench uses the audited local-input configuration below at 128×128 resolution. Both WAMs use 20 video- and 50 action-denoising steps and AdamW with learning rate 10^{-5} . Training uses batch size 1, gradient accumulation 10, and 5,000 steps on eight H200 GPUs.

DiM-WAM uses eight 12-slot banks that condition both denoising branches. We drop historical clean KV entries with probability 0.3 and truncate backpropagation through time to 4 segments.

Real-robot experiments use one third-person camera at 224×224 . The WAMs otherwise retain the simulation configuration. Fast-WAM [27] uses 150 epochs, cosine scheduling, learning rate 10^{-4} , weight decay 10^{-2} , and effective batch size 16. $\pi_{0.5}$ [3] uses its recommended 20K-step bfloat16 recipe with batch size 24. All methods predict seven absolute joint positions and one binary gripper state.

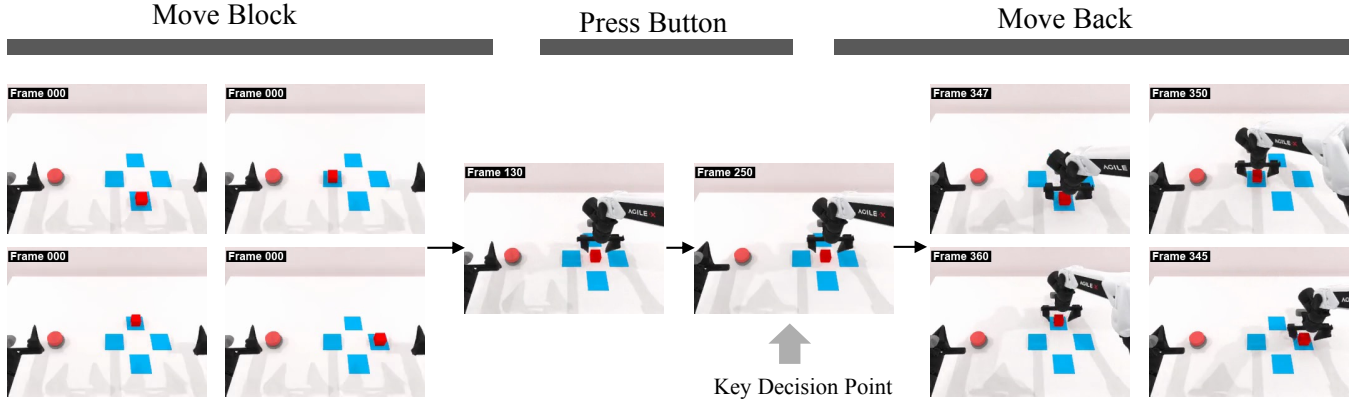


Fig. 5. Key frames of the RMBench `put_back_block` task. Different initial block locations lead to similar intermediate states, while the final move-back target still depends on the initial location that may fall outside the local observation window.

TABLE I
PROTOCOL AUDIT ON `PUT_BACK_BLOCK`. “CHANCE” IS THE IDEAL
TARGET-CHOICE RATE FROM LOCAL CONTEXT, NOT A FULL-TASK UPPER
BOUND.

Camera views	Window	Stride	Initial visible	Chance	Success
front + wrist + head	30	1	False	25	86
head + front	30	1	False	25	41
head + front	60	1	False	25	49
head + front	120	1	True	100	88
front + wrist + head	30	4	True	100	100
head + front	30	4	True	100	98

B. Evaluation-Protocol Audit

We audit whether camera views and temporal coverage expose the initial location to the local policy.

Wrist views raise success from 41% to 86% despite the initial state being outside the local window, suggesting pose-dependent leakage; we therefore use head+front third-person views, which vary less with arm pose. To isolate memory benefits, we avoid window enlargement and temporal subsampling, which recover the initial state (88%/98%), and keep the LingBot-VA defaults: window 30, stride 1, and four actions per video latent frame. † marks this setting in Table II. This audit was conducted on `put_back_block` as a representative $M(1)$ task; we did not repeat the full sweep for every task, so residual leakage in other RMBench tasks cannot be ruled out.

C. Simulation Experiments

Table II reports two types of evidence. First, it places DiM-WAM in the RMBench evaluation context: DP [41], ACT [42], $\pi_{0.5}$ [3], X-VLA [43], and Mem-0 [14] are benchmark-reported results under the same task definitions and success metrics. Second, it provides our controlled WAM comparison between LingBot-VA and DiM-WAM, which use identical local observations, demonstrations, camera views, window/stride configuration, and common training configuration apart from

the proposed memory/progress components and objective. Both WAMs use 50 demonstrations per task and 100 rollouts in the official environments with default seeds. We therefore use the LingBot-VA comparison for causal attribution and treat other baseline families as benchmark-level reference points because their training recipes and model interfaces may differ. Consequently, comparisons against DP, ACT, $\pi_{0.5}$, X-VLA, and Mem-0 in Table II should be read as contextual reference points rather than controlled ablations; only the LingBot-VA vs. DiM-WAM columns support causal claims about the proposed memory design.

Within the RMBench evaluation context, DiM-WAM reports the highest success on eight of nine tasks and exceeds benchmark-reported Mem-0 by 27.8 points overall, although Mem-0 remains strongest on Cover Blocks. More importantly, the controlled LingBot-VA comparison isolates the contribution of the proposed memory design: DiM-WAM improves overall success by 35.0 percentage points, from 34.8% to 69.8%; the gains are 46.4 points on $M(1)$ (34.2% to 80.6%) and 20.8 points on $M(n)$ (35.5% to 56.3%).

D. Real-World Experiments

We design four multi-stage Franka Panda tasks in which actions depend on prior spatial state, event order, or target identity (Fig. 6).

We compare $\pi_{0.5}$ [3], Fast-WAM [27], LingBot-VA [7], and DiM-WAM using 15–25 demonstrations and 10 trials per task. Table III reports completed-stage ratio (SSR) and full-task success (SR) as counts, with average columns kept as percentages.

DiM-WAM improves average SSR over LingBot-VA by 20.8 percentage points, from 70.7% to 91.5%, and average SR by 27.5 points, from 52.5% to 80.0%. The largest SR gains occur on Find Blue Block (+60.0 points; 10.0% to 70.0%) and Line Swap (+30.0 points; 60.0% to 90.0%), which require cross-stage identity or spatial memory.

$\pi_{0.5}$ and Fast-WAM achieve no full-task successes under this few-shot setting. On Press Twice, the local window

TABLE II

MAIN SIMULATION RESULTS ON RMBENCH LONG-HORIZON TASKS. VALUES ARE SUCCESS RATES IN PERCENT. TMC DENOTES THE RMBENCH $M(1)/M(n)$ TASK-MEMORY CATEGORY. DP, ACT, $\pi_{0.5}$, X-VLA, AND MEM-0 ARE BENCHMARK-REPORTED RESULTS. $\dagger$ MARKS OUR CONTROLLED WAM COMPARISON UNDER MATCHED INPUTS AND COMMON TRAINING SETTINGS APART FROM THE PROPOSED COMPONENTS.

Task	TMC	DP	ACT	$\pi_{0.5}$	X-VLA	Mem-0	LingBot-VA $\dagger$	DiM-WAM $\dagger$
Observe and Pick Up	$M(1)$	1.0	1.0	9.0	9.0	4.0	4.0	13.0
Rearrange Blocks	$M(1)$	0.0	29.0	13.0	13.0	89.0	63.0	99.0
Put Back Block	$M(1)$	0.0	0.0	11.0	18.0	90.0	41.0	98.0
Swap Blocks	$M(1)$	11.0	2.0	24.0	16.0	67.0	38.0	96.0
Swap T	$M(1)$	20.0	2.0	15.0	3.0	14.0	25.0	97.0
Average	$M(1)$	6.4	6.8	14.4	11.8	52.8	34.2	80.6
Battery Try	$M(n)$	10.0	19.0	16.0	26.0	28.0	33.0	48.0
Blocks Ranking Try	$M(n)$	10.0	0.0	6.0	1.0	18.0	48.0	87.0
Cover Blocks	$M(n)$	0.0	0.0	0.0	2.0	68.0	42.0	56.0
Press Button	$M(n)$	0.0	0.0	0.0	0.0	0.0	19.0	34.0
Average	$M(n)$	5.0	4.8	5.5	7.3	28.5	35.5	56.3
Total Average	—	5.8	5.9	10.4	9.8	42.0	34.8	69.8

TABLE III

REAL-WORLD FRANKA PANDA RESULTS. TASK ENTRIES ARE SUCCESS COUNTS OVER TOTAL ATTEMPTS. SSR DENOMINATORS ARE 40 FOR FIND BLUE BLOCK, 60 FOR EACH SWAP TASK, AND 30 FOR PRESS TWICE; ALL SR DENOMINATORS ARE 10. AVG. VALUES ARE PERCENTAGES.

Method	Find Blue Block		Line Swap		Triangle Swap		Press Twice		Avg. (%)	
	SSR	SR	SSR	SR	SSR	SR	SSR	SR	SSR	SR
$\pi_{0.5}$	0/40	0/10	3/60	0/10	0/60	0/10	0/30	0/10	1.3	0.0
Fast-WAM	0/40	0/10	0/60	0/10	0/60	0/10	0/30	0/10	0.0	0.0
LingBot-VA	21/40	1/10	47/60	6/10	31/60	4/10	30/30	10/10	70.7	52.5
DiM-WAM(ours)	37/40	7/10	57/60	9/10	47/60	6/10	30/30	10/10	91.5	80.0



Fig. 6. Real-world Franka Panda tasks. The four panels show Triangle Swap (top-left), Line Swap (bottom-left), Press Twice (top-right), and Find Blue Block (bottom-right).

already covers about 85% of each demonstration, and both WAMs reach 100.0%; memory therefore preserves local-control performance when long-term context is unnecessary.

TABLE IV

ABLATION STUDY ON MEMORY STRUCTURE AND PROGRESS SUPERVISION. VALUES ARE SUCCESS RATES IN PERCENT.

Variant	Swap T	Swap Blocks	Avg.
1×32 memory	71.0	80.0	75.5
4×8 memory	88.0	92.0	90.0
8×12 memory	97.0	96.0	96.5
8×12 w/o prog head	92.0	89.0	90.5

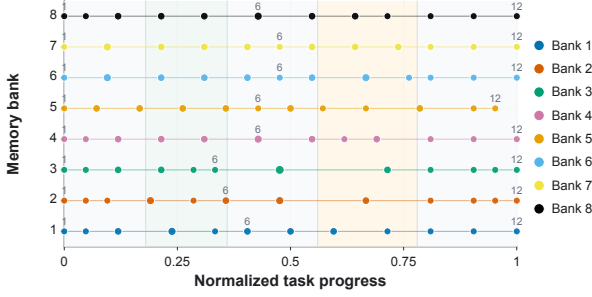
E. Ablation Studies

Table IV ablates bank structure and progress supervision on Swap T and Swap Blocks.

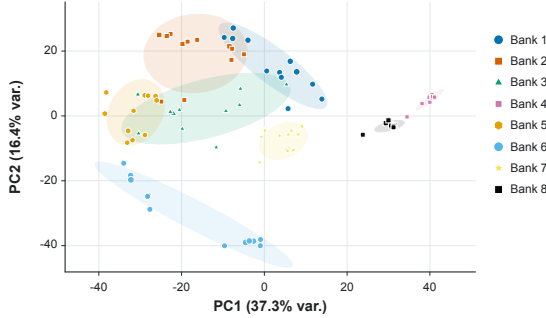
At the same 32-slot capacity, replacing one bank with four banks raises average success by 14.5 percentage points, from 75.5% to 90.0%. Increasing the configuration to 8×12 further reaches 96.5%, while removing progress supervision lowers it by 6.0 points to 90.5%.

F. Cross-Bank Behavior Analysis

We visualize retained events and token representations from one representative successful `put_back_block` episode (Fig. 7) as a qualitative illustration of whether bank-conditioned extraction leads to different compression trajectories.



(a) Retained memory events over task progress.



(b) PCA of memory tokens.

Fig. 7. Cross-bank behavior in one episode. (a) Retained events over normalized progress; rows denote banks, shading denotes task phases, and annotations show retained mass. (b) PCA of memory tokens, with colors, markers, and ellipses denoting bank-wise distributions.

Banks retain different event positions: some favor early or boundary events, whereas others preserve middle- or late-stage changes. Retained mass often concentrates near transitions, and PCA shows distinct but overlapping bank distributions. These differences arise from bank-specific queries and identity embeddings producing distinct observation summaries, followed by state-dependent, bank-local compression. While this single-episode analysis is not a quantitative claim about generality across episodes or tasks, it is qualitatively consistent with the complementary-representation trend suggested by the bank-structure ablation in Table IV.

V. CONCLUSION

DiM-WAM is a memory-augmented world-action model for long-horizon robot manipulation. It separates complementary observation summaries into fixed-capacity memory banks and supervises them with task progress, enabling the policy to retain early task evidence while predicting near-term dynamics.

Experiments show consistent gains in both simulation and the real world. In the controlled RMBench comparison, DiM-WAM improves average success from 34.8% to 69.8% over the LingBot-VA baseline, and on four Franka tasks it raises stage and full-task success from 70.7%/52.5% to

91.5%/80.0%. Ablations on two representative RMBench tasks indicate that multi-bank memory and progress supervision are beneficial in these settings.

Limitations and future work. The current study is still limited by non-exhaustive memory-bank ablations, 128×128 inputs, a head+front RMBench camera setting, and a relatively small set of real-robot trials. Results are reported as point estimates without seed-level variance or repeated real-robot evaluation rounds, and both LingBot-VA and RMBench are recent preprints, which may affect the stability of absolute comparisons. Future work will study matched-compute memory scaling, heterogeneous bank designs, higher-resolution and multi-view inputs, broader task and embodiment distributions, and more repeated real-world evaluations.

REFERENCES

- [1] A. Brohan *et al.*, “RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control,” in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 2165–2183.
- [2] M. J. Kim *et al.*, “OpenVLA: An Open-Source Vision-Language-Action Model,” in *Proceedings of the 8th Conference on Robot Learning*, 2024.
- [3] K. Black *et al.*, “ π_0 : A Vision-Language-Action Flow Model for General Robot Control,” in *Proceedings of Robotics: Science and Systems*, 2025.
- [4] S. Liu *et al.*, “RDT-1B: A Diffusion Foundation Model for Bimanual Manipulation,” in *International Conference on Learning Representations*, 2025.
- [5] NVIDIA *et al.*, “GR00T N1: An Open Foundation Model for Generalist Humanoid Robots,” *arXiv preprint arXiv:2503.14734*, 2025.
- [6] C.-L. Cheang *et al.*, “GR-2: A Generative Video-Language-Action Model with Web-Scale Knowledge for Robot Manipulation,” *arXiv preprint arXiv:2410.06158*, 2024.
- [7] L. Li *et al.*, “Causal World Modeling for Robot Control,” *arXiv preprint arXiv:2601.21998*, 2026.
- [8] S. Ye *et al.*, “World Action Models are Zero-Shot Policies,” *arXiv preprint arXiv:2602.15922*, 2026.
- [9] B. E. Sherman, N. B. Turk-Browne, and E. V. Goldfarb, “Multiple Memory Subsystems: Reconsidering Memory in the Mind and Brain,” *Perspectives on Psychological Science*, vol. 19, no. 1, pp. 103–125, 2024.
- [10] H. Fang *et al.*, “SAM2Act: Integrating Visual Foundation Model with A Memory Architecture for Robotic Manipulation,” in *International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267, 2025.
- [11] H. Shi *et al.*, “MemoryVLA: Perceptual-Cognitive Memory in Vision-Language-Action Models for Robotic Manipulation,” in *International Conference on Learning Representations*, 2026.
- [12] M. Torne *et al.*, “MEM: Multi-Scale Embodied Memory for Vision Language Action Models,” *arXiv preprint arXiv:2603.03596*, 2026.
- [13] H. Li *et al.*, “ReMem-VLA: Empowering Vision-Language-Action Model with Memory via Dual-Level Recurrent Queries,” *arXiv preprint arXiv:2603.12942*, 2026.
- [14] T. Chen *et al.*, “RMBench: Memory-Dependent Robotic Manipulation Benchmark with Insights into Policy Design,” *arXiv preprint arXiv:2603.01229*, 2026.
- [15] D. Qu *et al.*, “SpatialVLA: Exploring Spatial Representations for Visual-Language-Action Models,” in *Proceedings of Robotics: Science and Systems*, 2025.
- [16] P. Li *et al.*, “BridgeVLA: Input-Output Alignment for Efficient 3D Manipulation Learning with Vision-Language Models,” in *Advances in Neural Information Processing Systems*, 2025.
- [17] G. Yang *et al.*, “EventVLA: Event-Driven Visual Evidence Memory for Long-Horizon Vision-Language-Action Policies,” *arXiv preprint arXiv:2606.20092*, 2026.
- [18] H. Shi *et al.*, “MemoryVLA++: Temporal Modeling via Memory and Imagination in Vision-Language-Action Models,” *arXiv preprint arXiv:2606.09827*, 2026.

- [19] Y. Du *et al.*, “Learning Universal Policies via Text-Guided Video Generation,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [20] S. Zhou, Y. Du, J. Chen, Y. Li, D.-Y. Yeung, and C. Gan, “RoboDreamer: Learning Compositional World Models for Robot Imagination,” in *International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024.
- [21] H. Bharadhwaj *et al.*, “Gen2Act: Human Video Generation in Novel Scenarios Enables Generalizable Robot Manipulation,” *arXiv preprint arXiv:2409.16283*, 2024.
- [22] Y. Hu *et al.*, “Video Prediction Policy: A Generalist Robot Policy with Predictive Visual Representations,” in *International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267, 2025.
- [23] Y. Tian *et al.*, “Predictive Inverse Dynamics Models are Scalable Learners for Robotic Manipulation,” in *International Conference on Learning Representations*, 2025.
- [24] M. J. Kim *et al.*, “Cosmos Policy: Fine-Tuning Video Models for Visuomotor Control and Planning,” *arXiv preprint arXiv:2601.16163*, 2026.
- [25] T. Ma *et al.*, “DiT4DiT: Jointly Modeling Video Dynamics and Actions for Generalizable Robot Control,” *arXiv preprint arXiv:2603.10448*, 2026.
- [26] Y. Su *et al.*, “World Guidance: World Modeling in Condition Space for Action Generation,” *arXiv preprint arXiv:2602.22010*, 2026.
- [27] T. Yuan, Z. Dong, Y. Liu, and H. Zhao, “Fast-WAM: Do World Action Models Need Test-Time Future Imagination?” *arXiv preprint arXiv:2603.16666*, 2026.
- [28] A. Ye *et al.*, “GigaWorld-Policy: An Efficient Action-Centered World-Action Model,” *arXiv preprint arXiv:2603.17240*, 2026.
- [29] R. Henschel *et al.*, “StreamingT2V: Consistent, Dynamic, and Extendable Long Video Generation from Text,” *arXiv preprint arXiv:2403.14773*, 2024.
- [30] Y. Oshima, S. Taniguchi, M. Suzuki, and Y. Matsuo, “SSM Meets Video Diffusion Models: Efficient Long-Term Video Generation with Structured State Spaces,” in *ICLR Workshop*, 2024.
- [31] R. Po *et al.*, “Long-Context State-Space Video World Models,” *arXiv preprint arXiv:2505.20171*, 2025.
- [32] J. Chen *et al.*, “SANA-Video: Efficient Video Generation with Block Linear Diffusion Transformer,” in *International Conference on Learning Representations*, 2026.
- [33] S. Yu *et al.*, “MALT Diffusion: Memory-Augmented Latent Transformers for Any-Length Video Generation,” in *CVPR Workshop on AI for Content Creation*, 2025.
- [34] T. Zhu, S. Zhang, Z. Sun, J. Tian, and Y. Tang, “Memorize-and-Generate: Towards Long-Term Consistency in Real-Time Video Generation,” *arXiv preprint arXiv:2512.18741*, 2025.
- [35] Y. Yu *et al.*, “VideoSSM: Autoregressive Long Video Generation with Hybrid State-Space Memory,” *arXiv preprint arXiv:2512.04519*, 2025.
- [36] S. Chen *et al.*, “Context Forcing: Consistent Autoregressive Video Generation with Long Context,” *arXiv preprint arXiv:2602.06028*, 2026.
- [37] Z. Zhao, Y. Lu, Z. Liu, J. Song, J. Deng, and I. Patras, “Relax Forcing: Relaxed KV-Memory for Consistent Long Video Generation,” *arXiv preprint arXiv:2603.21366*, 2026.
- [38] S. Ji, X. Chen, S. Yang, X. Tao, P. Wan, and H. Zhao, “MemFlow: Flowing Adaptive Memory for Consistent and Efficient Long Video Narratives,” *arXiv preprint arXiv:2512.14699*, 2025.
- [39] J. Zhou *et al.*, “VideoMemory: Toward Consistent Video Generation via Memory Integration,” *arXiv preprint arXiv:2601.03655*, 2026.
- [40] W. Dou, H. Li, J. Cui, L. Zhou, J. Wang, and S. Zhu, “SlotMemory: Object-Centric KV Memory for Streaming Long-Video Generation,” *arXiv preprint arXiv:2605.31033*, 2026.
- [41] C. Chi *et al.*, “Diffusion Policy: Visuomotor Policy Learning via Action Diffusion,” in *Proceedings of Robotics: Science and Systems*, 2023.
- [42] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,” in *Proceedings of Robotics: Science and Systems*, 2023.
- [43] J. Zheng *et al.*, “X-VLA: Soft-Prompted Transformer as Scalable Cross-Embodiment Vision-Language-Action Model,” *arXiv preprint arXiv:2510.10274*, 2025.